Ranking and Unranking Permutations in Heap's Order

Yusheng HU[1*]

[1*]Independent Researcher, Shangdi East Road, Beijing, 100085, China.

Corresponding author(s). E-mail(s): dr.huyusheng@gmail.com;

Abstract

Heap's algorithm is a permutation generation method with very low constant overhead. Its element-interchanging mechanism, however, prevents the standard factorial-base Lehmer code from being applied directly, and to our knowledge no explicit Rank/Unrank algorithm for the Heap order has been published. This paper describes the block structure of Heap's algorithm in terms of rearrangement operators: the net effect of a block-internal traversal is a tail-preserving operator and the block-boundary swap is a top-level swap operator, and together they yield a closed-form recurrence for the block transitions. On this basis a bottom-up Unrank algorithm and a top-down Rank algorithm are built from one and the same closed-form in-block mapping; both run in $\boldsymbol{O(n^3)}$ time and $\boldsymbol{O(n)}$ space and require no factorial-scale precomputed table. The supporting proofs use a synchronized strong induction on the dimension to establish the bijectivity of the trailing-element mapping, closed forms for the final arrangement and for the trailing sequence, the dichotomy of its cycle structure, and the reciprocity of Rank and Unrank. The Heap order is in fact the colex order of the mixed-radix code.

Keywords: Heap's Algorithm; Permutation Generation; Rank; Unrank; Factorial Base; Operator Algebra; Fractal Self-Similarity

MSC Classification: 68Q25 , 05A05

1 Introduction and Related Work

The indexing and mapping of permutations involve two basic operations: **Rank** and **Unrank**. For a fixed generation order, Rank takes a permutation as input and returns its index, whereas Unrank takes an index and returns the permutation. When the index

---

No funding was received for this study.

is written as a factorial-base Lehmer code, the two operations amount to mutually inverse maps between the set of permutations and the code space.

Heap's algorithm (1963) is widely used for permutation enumeration because of its in-place, minimal-interchange character [2, 5]; Knuth also gives a detailed account of the method and of its permutation-theoretic properties [8]. The Heap order is a minimal-change generation order: consecutive permutations differ by a single transposition, not necessarily of adjacent positions. Among orders of this kind, the construction of Johnson and Trotter uses adjacent transpositions only [1, 3], and Ives gave several further interchange-based enumeration schemes [4]. The standard Heap algorithm uses an iterative interchange logic that partitions the $n!$ permutation space into n disjoint sub-blocks of $(n-1)!$ permutations each: inside a sub-block the first $n-1$ positions traverse the lower-order permutations while the trailing element stays locally fixed, and at a block boundary a single interchange updates the trailing element and moves on to the next sub-block. This block structure is recursively self-similar, but the interchange rule at a block transition alternates with the parity of the dimension, so the standard lexicographic Lehmer-code formula cannot be applied directly. To our knowledge, no explicit Rank/Unrank algorithm for the Heap order has been published.

As for related work, Myrvold and Ruskey (2001) gave a linear-time general permutation rank/unrank algorithm [6], but it is based on an arbitrarily associated mixed-radix encoding and has no direct correspondence with the block-transition structure of the Heap order. A systematic account of generation algorithms and of the associated rank/unrank techniques can be found in Ruskey's draft book [7].

The paper describes the block structure of Heap's algorithm through the rearrangement operators that realise it, and uses that description to obtain an index for the Heap order. With a source-index convention fixing the composition direction, a block-internal traversal has the net effect of the tail-preserving operator $\Lambda_{\mathcal{F}_{m-1}}$ and a block boundary is crossed by the top-level swap operator $\widehat{\sigma}_{m,q+1}$, so the block base arrays satisfy a closed-form recurrence. Both a bottom-up Unrank algorithm and a top-down Rank algorithm are built on that recurrence; they share the same operators, need no precomputed table, and run in $O(n^3)$ time and $O(n)$ space. The recurrence and the algorithms are justified by a strong induction on the dimension (Section 5), which also gives the closed form of the final arrangement and of the trailing sequence.

2 Preliminaries

2.1 Iterative Heap's Algorithm and Block Structure

The standard iterative Heap algorithm takes an initial array and an all-zero counter array as inputs and generates all permutations through cyclic interchanges. Its pseudocode is as follows.

```
while i < n:
    if c[i] < i:
        if i is even: swap(A[0], A[i])
        if i is odd:  swap(A[c[i]], A[i])
        output current permutation A
```

```
        c[i] += 1
        i = 0
    else:
        c[i] = 0
        i += 1
```

The algorithm has a recursive block property: the $n!$ permutation space is divided into n sub-blocks of $(n-1)!$ permutations each.

1. Inside a sub-block the first $n-1$ positions traverse all Heap permutations of order $n-1$, while the last element of the array stays fixed.
2. Once a sub-block has been traversed completely, a cross-block interchange updates the trailing element and enters the next sub-block.

Let V_q denote the trailing value at the beginning of the q-th block. By a property of the block structure proved below, $V_0, V_1, \ldots, V_{m-1}$ form a permutation of $\{0, 1, \ldots, m-1\}$, that is, no two blocks begin with the same trailing value (Lemma 4).

For a precise statement later on, write the output sequence of the order-m algorithm started from the identity array I_m as

$$A_0^{(m)},\ A_1^{(m)},\ \ldots,\ A_{m!-1}^{(m)}, \qquad A_0^{(m)} = I_m,$$

and define the **block base array** (the first arrangement of the q-th block) and its **trailing element** by

$$C_{m,q} := A^{(m)}_{q\,(m-1)!}, \qquad V_q := C_{m,q}[m-1] \qquad (0 \le q \le m-1),$$

together with the **final arrangement** of the whole run, $\mathcal{F}_m := A^{(m)}_{m!-1}$. In what follows, abstract statements at the operator level use m for the dimension, while statements about the algorithms use n; the two mean the same.

2.2 Arrangement Space and Operator Conventions

Let $S_m := \mathrm{Sym}(\{0, 1, \ldots, m-1\})$ be the symmetric group on m elements, whose members $P \in S_m$ are position permutations $P : \{0, 1, \ldots, m-1\} \to \{0, 1, \ldots, m-1\}$.

Definition 1 (Arrangement array set) The set of arrangement arrays of order m is

$$\mathrm{Arr}_m = \{[a_0, a_1, \ldots, a_{m-1}] \mid \{a_0, a_1, \ldots, a_{m-1}\} = \{0, 1, \ldots, m-1\}\}.$$

Its cardinality is $|\,\mathrm{Arr}_m\,| = m!$. The identity array is $I_m = [0, 1, \ldots, m-1] \in \mathrm{Arr}_m$.

Definition 2 (Rearrangement operator group, source-index convention) The rearrangement operator group Op_m is induced by position permutations. For $P \in S_m$ the induced rearrangement operator $\widehat{P} \in \mathrm{Op}_m$ acts on an array $A \in \mathrm{Arr}_m$ **under the source-index convention**

$$\widehat{P}(A)[i] = A[P(i)], \quad \forall i \in \{0, 1, \ldots, m-1\}.$$

Thus $P(i)$ is the **position** in the original array A whose entry moves to the new index i. If the permutation is a 2-transposition $(i\ j)$, the operator $\widehat{(i\ j)} \in \mathrm{Op}_m$ is called a **position interchange operator**; its action exchanges the entries at indices i and j of the current array.

Remark 1 (Meaning of the source-index convention) For $A = [a_0, a_1, a_2]$ and the transposition $P = (0\ 1)$, the convention gives

$$\widehat{(0\ 1)}(A) = [A[1], A[0], A[2]] = [a_1, a_0, a_2],$$

which is the statement `swap(A[0], A[1])` in program code. Operators compose by ordinary function composition, $(\alpha \circ \beta)(A) = \alpha(\beta(A))$.

Lemma 1 (Operator anti-homomorphism) *Under the source-index convention the map $P \mapsto \widehat{P}$ is a group isomorphism from the opposite group S_m^{op} to Op_m, that is,*

$$\widehat{P} \circ \widehat{Q} = \widehat{Q \circ P} \qquad \text{for all } P, Q \in S_m.$$

Proof For any $A \in \mathrm{Arr}_m$ and any index $i \in \{0, 1, \ldots, m-1\}$, expanding layer by layer under the source-index convention gives

$$(\widehat{P} \circ \widehat{Q})(A)[i] = \widehat{P}\big(\widehat{Q}(A)\big)[i] = \widehat{Q}(A)\big[P(i)\big] = A\big[Q(P(i))\big] = A\big[(Q \circ P)(i)\big] = \widehat{Q \circ P}(A)[i].$$

Hence the anti-homomorphism formula holds. Since Op_m is by definition induced by S_m, the map $P \mapsto \widehat{P}$ is a bijection, so it is an isomorphism $S_m^{\mathrm{op}} \to \mathrm{Op}_m$. □

Definition 3 (Canonical isomorphism between arrays and permutations) For an array $A = [a_0, a_1, \ldots, a_{m-1}] \in \mathrm{Arr}_m$, its induced position permutation $\pi_A \in S_m$ is defined by $\pi_A(i) = a_i$; then $A = \widehat{\pi_A}(I_m)$.

2.3 Factorial-Base Code Space

A permutation index $r \in [0, n!-1]$ has a unique decomposition into a **low-order-first** mixed-radix (factorial-base) code

$$d = (d_0, d_1, \ldots, d_{n-2}), \qquad 0 \le d_i \le i+1,$$

whose weights increase from left to right:

$$r = \sum_{i=0}^{n-2} d_i \cdot (i+1)! = d_0 \cdot 1! + d_1 \cdot 2! + \cdots + d_{n-2} \cdot (n-1)!.$$

The code space is therefore $\prod_{i=0}^{n-2}\{0, 1, \ldots, i+1\}$, of cardinality $\prod_{i=0}^{n-2}(i+2) = n!$, the same as Arr_n. **The pseudocode, the numerical tables and all theorems below use this code** d**.** The most significant digit d_{n-2} is the block index, as proved by Lemma 2 and Theorem 7: $d_{n-2} = q$ selects the sub-block containing the permutation,

d_{n-3} selects the sub-block of the corresponding $(n-1)$-order sub-permutation in the lower-order space, and so on recursively downwards, which is the basis of the hierarchical mapping.

Remark 2 (Relation to the standard Lehmer code) The **high-order-first Lehmer code** $C = (C_0, C_1, \ldots, C_{n-1})$ common in the literature orders the weights in decreasing fashion and is the **index reversal** of the code d used here:

$$C_j = d_{n-2-j} \quad (0 \le j \le n-2), \qquad C_{n-1} = 0, \qquad \text{equivalently} \quad d_i = C_{n-2-i}.$$

The constraint $0 \le d_i \le i+1$ becomes the standard Lehmer constraint $0 \le C_j \le n-1-j$, and the two writings give the same index

$$r = \sum_{i=0}^{n-2} d_i \cdot (i+1)! = \sum_{j=0}^{n-2} C_j \cdot (n-1-j)!.$$

With the low-order-first reading, the Heap generation order is the **colex order** of the code vectors rather than their lexicographic order. This is why the lexicographic Lehmer formula cannot be applied to the Heap order, and it accounts for the shape $\sum_i d_i(i+1)!$ of the rank formula.

3 Fractal Self-Similar Mapping Theory and Operator Systems

3.1 Position-Value Mapping and Inter-Block Rearrangement

Define the **position–value mapping** as follows: the input is a position index of the sub-array and the output is the value stored there. For one block of the order-m algorithm the domain is $\{0, 1, \ldots, m-2\}$ (the first $m-1$ positions) and the range is the set of values other than the current trailing value; the initial state is the identity mapping, with trailing value $m-1$.

Definition 4 (Position–value mapping versus block base arrays) For the q-th block of the order-m algorithm $(0 \le q \le m-1)$, define its **position–value mapping** S_q and its **trailing value** ℓ_q by

$$S_q(i) := C_{m,q}[i] \quad (0 \le i \le m-2), \qquad \ell_q := C_{m,q}[m-1] = V_q.$$

Thus S_q is just the "position $\to$ value" reading of the first $m-1$ entries of the block base array $C_{m,q}$, and the trailing value is its m-th entry. The array-level objects $(C_{m,q}, V_q)$ of Section 2.1 and the mapping-level objects (S_q, ℓ_q) of this section are therefore two views of the same state: the former is convenient for writing operators, the latter for stating the closure and invertibility of the mapping.

Remark 3 (Block-boundary position–value rearrangement) Suppose a block has been traversed completely and the algorithm is about to switch to the next block, with position–value mapping S_q and trailing value ℓ_q. The pseudocode shows that the switch interchanges the trailing value with the value at one prescribed position: the old value at that position becomes

the new trailing value, and the old trailing value fills that position. The position is determined by the parity rule: for even m the target is $\mathrm{pos} = q$ (where q is the current block index, i.e. the block being left), and for odd m it is always $\mathrm{pos} = 0$.

At the mapping level, therefore, a block switch is a single interchange at a prescribed position, while the net effect of the traversal within a block is a fixed permutation; the two together give the initial mapping of the next block. The rigorous form of this statement, and its entrywise correspondence with the top-level swap operator $\widehat{\sigma}_{m,q}$ and the tail-preserving operator Λ_A, is given by Lemma 2 in Section 3.3. For odd order the cyclic-shift description is not closed when its prefix has length 2 (that is, $m = 3$) and degenerates into a transposition, so the closed form needs degenerate treatment for $m \leq 3$ (see Section 4.4).

3.2 Top-Level Swap and Tail-Preserving Operators

Definition 5 (Top-level swap operator $\widehat{\sigma}_{m,q}$) For an array of size m, with block index parameter $q \in \{0, 1, \ldots, m-1\}$, the top-level swap operator $\widehat{\sigma}_{m,q} \in \mathrm{Op}_m$ is

$$\widehat{\sigma}_{m,q} = \begin{cases} \mathrm{id}, & q = 0, \\ \widehat{(0\ m-1)}, & m \text{ odd and } 1 \leq q \leq m-1, \\ \widehat{(q-1\ m-1)}, & m \text{ even and } 1 \leq q \leq m-1. \end{cases}$$

Definition 6 (Tail-preserving operator Λ_A) Let $\pi \in \mathrm{Sym}(\{0, \ldots, m-2\})$ and let the monomorphic embedding E_m extend it to an order-m permutation that fixes position $m-1$. For an array $A \in \mathrm{Arr}_{m-1}$ of order $m-1$, the associated tail-preserving rearrangement operator is $\Lambda_A := \widehat{E_m(\pi_A)} \in \mathrm{Op}_m$; its action on an m-dimensional array X is

$$\Lambda_A(X) = [x_{\pi_A(0)}, x_{\pi_A(1)}, \ldots, x_{\pi_A(m-2)},\ x_{m-1}].$$

That is, the first $m-1$ entries are rearranged according to the source indices of the permutation induced by A, while the trailing entry x_{m-1} is left untouched.

3.3 Block Transition Operator and Block Base Arrays

Definition 7 (Block transition operator $B_{m,q}$) Define $B_{m,q} \in \mathrm{Op}_m$ by

$$B_{m,0} = \mathrm{id}, \qquad B_{m,q+1} = \widehat{\sigma}_{m,q+1} \circ \Lambda_{\mathcal{F}_{m-1}} \circ B_{m,q} \quad (0 \leq q \leq m-2).$$

Here $\mathcal{F}_{m-1}$ is the final arrangement of the order-$(m-1)$ algorithm defined in Section 2.1, so the recursion is well founded ($m - 1 < m$).

Lemma 2 (Block self-similarity and realizability of the block transition operator) *For every $0 \leq q \leq m-2$ the block base arrays $C_{m,q}$ of the order-m algorithm satisfy the block recurrence*

$$C_{m,q+1} = \widehat{\sigma}_{m,q+1}\big(\Lambda_{\mathcal{F}_{m-1}}(C_{m,q})\big).$$

so that $C_{m,q} = B_{m,q}(I_m)$ for $0 \leq q \leq m-1$, and

$$\mathcal{F}_m = \Lambda_{\mathcal{F}_{m-1}}(C_{m,m-1}).$$

The correspondence holds output by output as well: for $0 \leq k \leq (m-1)! - 1$,

$$A^{(m)}_{q\,(m-1)!+k} = E_m\big(\widehat{\pi_{A^{(m-1)}_k}}\big)\big(C_{m,q}\big),$$

where E_m is the embedding of Section 3.2. Thus the k-th output inside the q-th block is the block base array rearranged by the position permutation carried by the k-th permutation of order $m-1$, and $B_{m,q}$ reproduces the block-level transitions of the iterative Heap algorithm.

Proof The indices involved in each interchange depend only on the dimension m and on the counter c, never on the values held in the array. For a fixed m the algorithm therefore performs the same sequence of interchange positions on any initial array, and the net effect of any finite segment of interchanges is the action of one fixed position permutation, that is, an operator of the form $\widehat{\sigma}$ (by Lemma 1, a composition of position transpositions is again of this form).

Consider one block. When i increases to $m-1$, the counters $c[0], \ldots, c[m-2]$ have all been reset to zero in the course of the run, so the operations on positions $0, \ldots, m-2$ amount to one complete Heap process of order $m-1$ with the counters restarted from zero, while the interchange executed at $i = m-1$ is the cross-block interchange. Since the q-th block contains $(m-1)!$ outputs, exactly $(m-1)! - 1$ further outputs follow the block base array before that interchange occurs, so the prefix process runs exactly once per block.

Let $\widehat{\sigma}$ with $\sigma \in S_{m-1}$ be the net effect of one complete process of order $m-1$ on an arbitrary prefix array X. As the interchange positions are value-independent, σ does not depend on X; taking $X = I_{m-1}$ shows that $\widehat{\sigma}(I_{m-1})$ is the final arrangement $\mathcal{F}_{m-1}$ of the order-$(m-1)$ process, so $\sigma(i) = \mathcal{F}_{m-1}[i] = \pi_{\mathcal{F}_{m-1}}(i)$. The process therefore acts on positions $0, \ldots, m-2$ through the tail-preserving operator $\Lambda_{\mathcal{F}_{m-1}}$, the trailing position $m-1$ being fixed throughout.

The cross-block interchange is the operator $\widehat{\sigma}_{m,q+1}$. Before entering block $q+1$ we have $c[m-1] = q$; for odd m the pseudocode performs $\mathrm{swap}(A[0], A[m-1]) = (0\ \widehat{m-1})$, and for even m it performs $\mathrm{swap}(A[q], A[m-1]) = (q\ \widehat{m-1})$, in agreement with the definition of $\widehat{\sigma}_{m,q+1}$.

The same reasoning applies step by step inside a block. Let $\sigma_k \in S_{m-1}$ be the partial net permutation after k steps of the prefix process ($0 \le k \le (m-1)! - 1$). It does not depend on the initial prefix state of the block, and applied to I_{m-1} the state after k steps is the k-th output $A_k^{(m-1)}$ of the order-$(m-1)$ algorithm, so $\sigma_k = \pi_{A_k^{(m-1)}}$; since the trailing position $m-1$ is fixed throughout the prefix process, this is the output-by-output formula.

These observations give the block recurrence. Induction on q, together with $C_{m,0} = A_0^{(m)} = I_m$, yields $C_{m,q} = B_{m,q}(I_m)$; and after the last block $q = m-1$ there is no further cross-block interchange, its prefix process running to the end of the whole run, whence $\mathcal{F}_m = \Lambda_{\mathcal{F}_{m-1}}(C_{m,m-1})$. □

3.4 Real-Time Derivation Rule for Trailing Values

The trailing value at the beginning of a block need not be generated and stored in advance: it is obtained in real time from the old value overwritten by the interchange of the previous step. During an interchange the old value stored at the prescribed position is taken as the new trailing value, while the previous trailing value fills that position. This property is the reason the algorithms below avoid factorial-scale lookup tables and keep only linear space.

3.5 Completeness of the Mapping Range

Proposition 3 (Completeness and invertibility of the position–value mapping) *For every block q ($0 \le q \le m-1$) the position–value mapping S_q and the trailing value ℓ_q satisfy the disjoint-union condition*

$$\mathrm{Range}(S_q) \cup \{\ell_q\} = \{0, 1, \ldots, m-1\},$$

and S_q is invertible.

Proof By definition $C_{m,q} \in \mathrm{Arr}_m$ is a permutation of $\{0, 1, \ldots, m-1\}$, so its first $m-1$ entries — that is, the range of S_q — are $m-1$ distinct elements, and the m-th entry $\ell_q = V_q$ is the only remaining element; the disjoint-union condition follows. Since S_q takes distinct values it is injective, so its inverse S_q^{-1} (looking up a position by value) exists. This is what allows the Rank algorithm to reconstruct sub-permutations backwards, layer by layer, without ambiguity. □

3.6 Unrank Combinator and Mapping

Definition 8 (Unrank combinator and permutation mapping) Let $d = (d_0, d_1, \ldots, d_{m-2})$ be a mixed-radix code.

1. For $m = 2$: $\widehat{\mathcal{U}}_2(0) = \mathrm{id}$ and $\widehat{\mathcal{U}}_2(1) = \widehat{(0\ 1)}$.
2. For $m \geq 3$:
$$\widehat{\mathcal{U}}_m(d_0, \ldots, d_{m-2}) = \Lambda_{A_{m-1}} \circ B_{m,d_{m-2}},$$
$$A_{m-1} = \mathrm{Unrank}_{m-1}(d_0, \ldots, d_{m-3}).$$
3. The array mapping is $\mathrm{Unrank}_m(d) := \widehat{\mathcal{U}}_m(d)(I_m)$.

4 Bidirectional Rank and Unrank Algorithms

By Lemma 2 a block transition uses only two operators: the net effect of the in-block traversal, $\Lambda_{\mathcal{F}_{k-1}}$, and the block-boundary interchange, $\widehat{\sigma}_{k,q+1}$. If the former is realized, via the closed form of Theorem 5, as a single source-index table $\mathrm{Step}(k)$, then the two algorithms can share one and the same recurrence, and no separate branch for $n \leq 3$ is needed.

4.1 In-Block Net Mapping Step(k)

$\mathrm{Step}(k)$ is the source-index table of the tail-preserving operator $\Lambda_{\mathcal{F}_k}$: for an array X of length k it performs $X \leftarrow [X[t_0], \ldots, X[t_{k-1}]]$, where $t = \mathrm{Step}(k)$. By Theorem 5, t is exactly the final arrangement $\mathcal{F}_k$ of order k, so $\mathrm{Step}(k)$ can be written down from the closed form in $O(k)$ time (Algorithm 1).

Algorithm 1 In-block net mapping Step(k)

Require: Order $k \geq 1$
Ensure: Source-index table $t = (t_0, \ldots, t_{k-1})$ with $X[i] \leftarrow X[t_i]$

if $k = 1$ **then**
    return $[0]$
else if $k = 2$ **then**
    return $[1, 0]$
end if
$t \leftarrow [0, 1, \ldots, k-1]$
if k is even **then**
    $t_0 \leftarrow k-3, \quad t_1 \leftarrow k-2$
    for $i = 2$ **to** $k-3$ **do**
        $t_i \leftarrow i-1$
    end for
    $t_{k-2} \leftarrow k-1, \quad t_{k-1} \leftarrow 0$
else
    $t_0 \leftarrow k-1$
    for $i = 1$ **to** $k-2$ **do**
        $t_i \leftarrow i$
    end for
    $t_{k-1} \leftarrow 0$
end if
return t

4.2 Unrank Algorithm: Bottom-Up Incremental Expansion

The input is the dimension together with the low-order-first code d, and the output is the corresponding permutation of the Heap sequence. The algorithm starts from the seed [0] at $k = 1$ and expands up to $k = n$: at level k it reads the digit d_{k-2} (the block index b), advances the position–value mapping S of that block from block 0 to block b, then uses S to translate the symbolic permutation obtained at the previous level into actual values and appends the trailing value.

Algorithm 2 Heap permutation unranking

Require: Dimension n, low-order-first code array $d = (d_0, \ldots, d_{n-2})$
Ensure: Heap permutation perm

perm $\leftarrow [0]$
for $k = 2$ **to** n **do**
    $b \leftarrow d_{k-2}, \quad S \leftarrow [0, 1, \ldots, k-2], \quad \text{last} \leftarrow k-1$
    for $j = 0$ **to** $b - 1$ **do**
        $S \leftarrow \text{Step}(k-1)(S)$
        if k is even **then**
            pos $\leftarrow j$
        else
            pos $\leftarrow 0$
        end if
        last, $S[\text{pos}] \leftarrow S[\text{pos}], \text{last}$
    end for
    perm $\leftarrow [S[x] \mid x \in \text{perm}] + [\text{last}]$
end for
return perm

4.3 Rank Algorithm: Top-Down Layer-by-Layer Reduction

The algorithm runs downwards from $k = n$ to $k = 2$. At level k it advances S and the trailing value step by step with Step until the trailing entry of the current permutation agrees with last; the number of steps is the block index b. It then strips the trailing element with S^{-1} and recovers the sub-permutation of one lower order.

Algorithm 3 Heap permutation ranking

Require: Dimension n, Heap permutation perm
Ensure: Low-order-first code array $d = (d_0, \ldots, d_{n-2})$

if $n = 1$ **then**
  return [ ]
end if
$p \leftarrow$ perm, $\quad d \leftarrow [0]^{n-1}$
for $k = n$ **downto** 2 **do**
  $S \leftarrow [0, 1, \ldots, k-2]$, $\quad$ last $\leftarrow k-1$, $\quad b \leftarrow 0$
  while $p[k-1] \neq$ last **do**
    $S \leftarrow \mathrm{Step}(k-1)(S)$
    if k is even **then**
      pos $\leftarrow b$
    else
      pos $\leftarrow 0$
    end if
    last, $S[\mathrm{pos}] \leftarrow S[\mathrm{pos}]$, last
    $b \leftarrow b + 1$
  end while
  $d_{k-2} \leftarrow b$
  $p \leftarrow [S^{-1}(p[i]) \mid 0 \leq i \leq k-2]$
end for
return d

4.4 Degenerate Cases and Termination

The piecewise form of Step(k) needs attention for $k \leq 4$. For $k = 1$ we have $t = [0]$, the identity mapping. For $k = 2$ the closed form of the even branch would give $t_0 = k - 3 = -1$, which is out of range, so Algorithm 1 takes $[1, 0]$ separately — this is why k starts at 2 in Algorithms 2 and 3. For $k = 3$ the odd branch gives $t = [2, 1, 0]$; the cyclic shift has degenerated into a transposition here, but the closed form remains valid. For $k = 4$ the middle segment of the even branch is empty and the form gives $t = [1, 2, 3, 0]$ directly. All small-order degeneracies are thus absorbed by the first two lines and the parity branches of Step, and the two main algorithms need no extra branch for $n \leq 3$.

The while loop of Algorithm 3 terminates after at most $k-1$ steps, because Lemma 4 makes the trailing sequence $V_0, \ldots, V_{k-1}$ a permutation of $\{0, \ldots, k-1\}$, so $p[k-1]$ eventually equals last; and the inverse S^{-1} used there exists by Proposition 3.

5 Core Theorems and Proofs

The results below are proved by a strong induction on the dimension m that carries three statements along at once: the block-base and trailing-value properties of Lemma 4, the closed form of the final arrangement in Theorem 5, and the cycle dichotomy

of Lemma 6. For each m they are established in that order, each using only the corresponding statements for smaller dimensions, so the induction is well founded.

5.1 Base Cases

Before the induction proper we verify the boundary and low-order cases explicitly.

- $m = 2$**.** The code space has a single digit $d_0 \in \{0, 1\}$. Initially $C_{2,0} = [0, 1]$ with trailing value $V_0 = 1$; for $d_0 = 1$ the operator $\widehat{\sigma}_{2,1} = \widehat{(0\ 1)}$ gives $C_{2,1} = [1, 0]$ with $V_1 = 0$. The final arrangement is $\mathcal{F}_2 = [1, 0]$, and $\pi_{\mathcal{F}_2} = (0\ 1)$ is a single 2-cycle, that is, a transposition.
- $m = 3$**.** By Table 1 of Section 6 the odd-order single-cycle orbit operator $C_{3,q} = \widehat{\rho}^{\,q}(I_3)$ holds for $q = 0, 1, 2$; the trailing sequence is $V = (2, 1, 0)$, the final arrangement is $\mathcal{F}_3 = [2, 1, 0]$, and $\pi_{\mathcal{F}_3} = (0\ 2)$ is a transposition.
- $m = 4$**.** By Table 2 of Section 6 the even-order block base arrays are $C_{4,0} = [0, 1, 2, 3]$, $C_{4,1} = [3, 1, 0, 2]$, $C_{4,2} = [0, 2, 3, 1]$ and $C_{4,3} = [3, 2, 1, 0]$, with trailing sequence $V = (3, 2, 1, 0)$. The final arrangement is $\mathcal{F}_4 = [1, 2, 3, 0]$, and $\pi_{\mathcal{F}_4}$ is the single 4-cycle $0 \mapsto 1 \mapsto 2 \mapsto 3 \mapsto 0$.

These base cases form the starting point of the strong induction.

5.2 Tail Element Bijection and State Invariants

Lemma 4 (Tail-element bijection and state invariants) *Let $C_{m,q} \in \mathrm{Arr}_m$ be the q-th block base array ($0 \le q \le m - 1$) with trailing element $V_q = C_{m,q}[m - 1]$. Then $q \mapsto V_q$ is a bijection of $\{0, 1, \ldots, m - 1\}$, and the following closed forms hold.*

1. ***Even order*** *($m \ge 4$).* *The trailing sequence has the closed form*
$$V = (m - 1,\ m - 2,\ 1,\ 2,\ \ldots,\ m - 3,\ 0),$$
and the terminal block base array is
$$C_{m,m-1} = [m - 1,\ m - 2,\ 1,\ 2,\ \ldots,\ m - 4,\ m - 3,\ 0].$$

2. ***Odd order*** *($m \ge 5$).* *Let $\pi_{\mathcal{F}_{m-1}}$ be a single $(m-1)$-cycle, put $c_0 = 0$ and $c_k = \pi^k_{\mathcal{F}_{m-1}}(0)$ for $1 \le k \le m - 2$, and define the m-cycle*
$$\rho := (\,0,\ m - 1,\ c_1,\ \ldots,\ c_{m-2}\,).$$
Then the block base arrays satisfy the explicit orbit formula $C_{m,q} = \widehat{\rho}^{\,q}(I_m)$, that is, $C_{m,q}[i] = \rho^q(i)$, whence $V_q = \rho^q(m - 1)$ with the explicit values $V_0 = m - 1$, $V_q = c_q$ for $1 \le q \le m - 2$, and $V_{m-1} = 0$. The trailing sequence therefore has the closed form
$$V = (m - 1,\ m - 4,\ m - 5,\ \ldots,\ 2,\ 1,\ m - 3,\ m - 2,\ 0),$$
where $m - 4, m - 5, \ldots, 1$ is the consecutive segment decreasing from $m - 4$ to 1. (The degenerate case $m = 3$ is verified among the base cases of Section 5.1.)

Proof By the induction hypothesis, when m is even, $m-1$ is odd and Lemma 6 gives $\pi_{\mathcal{F}_{m-1}} = (0\ \ m - 2)$ as a transposition; when m is odd, $m - 1$ is even and the same lemma gives that $\pi_{\mathcal{F}_{m-1}}$ is a single $(m - 1)$-cycle. On the other hand, Lemma 2 gives the block recurrence
$$C_{m,q+1} = \widehat{\sigma}_{m,q+1}\big(H_q\big), \qquad H_q = \Lambda_{\mathcal{F}_{m-1}}(C_{m,q}), \qquad C_{m,0} = I_m.$$

1. Even order ($m \geq 4$). Here $\widehat{\sigma}_{m,q+1} = (\widehat{q\ m-1})$ for $0 \leq q \leq m-2$, so only the indices q and $m-1$ are exchanged. Since $\pi_{\mathcal{F}_{m-1}} = (0\ m-2)$, the operator $\Lambda_{\mathcal{F}_{m-1}}$ exchanges only the indices 0 and $m-2$ and leaves every other position fixed.

The proof uses the following invariant for $1 \leq q \leq m-2$:

$$C_{m,q}[i] = \begin{cases} 0, & i = 0 \text{ and } q \text{ even}, \\ m-1, & i = 0 \text{ and } q \text{ odd}, \\ V_i, & 1 \leq i \leq q-1, \\ i, & q \leq i \leq m-3, \\ m-1, & i = m-2 \text{ and } q \text{ even}, \\ 0, & i = m-2 \text{ and } q \text{ odd}, \\ V_q, & i = m-1, \end{cases}$$

and assert at the same time that

$$V_0 = m-1, \qquad V_1 = m-2, \qquad V_{q+1} = q \ \ (1 \leq q \leq m-3), \qquad V_{m-1} = 0.$$

The two entries of the invariant that concern the indices 0 and $m-2$ make up the **oscillation law** (the value 0 alternates between these two indices according to the parity of q, and migrates to the trailing position in the last block), while the entry $C_{m,q}[i] = i$ for $q \leq i \leq m-3$ is the **position-freezing property** (for $q \leq j$ the value at index j is constantly j).

Base step ($q = 0 \to q = 1$). Since $C_{m,0} = I_m$ we have $V_0 = m-1$. Here

$$H_0 = [\, m-2,\ 1,\ 2,\ \ldots,\ m-3,\ 0,\ m-1\,].$$

Applying $\widehat{\sigma}_{m,1} = (\widehat{0\ m-1})$ gives

$$C_{m,1}[0] = H_0[m-1] = m-1, \qquad C_{m,1}[m-1] = H_0[0] = m-2,$$

with all other positions unchanged, that is, $C_{m,1} = [\, m-1, 1, 2, \ldots, m-3, 0, m-2\,]$ and $V_1 = m-2$. This agrees with the invariant for $q = 1$ (odd).

Inductive step ($q \to q+1$, $1 \leq q \leq m-3$). Since $q \leq m-3$, the freezing property gives $H_q[q] = C_{m,q}[q] = q$ (because $q \notin \{0, m-2\}$). Hence

$$V_{q+1} = C_{m,q+1}[m-1] = H_q[q] = q, \qquad C_{m,q+1}[q] = H_q[m-1] = C_{m,q}[m-1] = V_q.$$

As $\Lambda_{\mathcal{F}_{m-1}}$ exchanges only the indices 0 and $m-2$, we have $H_q[0] = C_{m,q}[m-2]$ and $H_q[m-2] = C_{m,q}[0]$; and since $q \leq m-3 < m-2$, the subsequent top-level swap $\widehat{\sigma}_{m,q+1} = (\widehat{q\ m-1})$ touches neither index 0 nor index $m-2$. Therefore:

- if q is even, then $C_{m,q}[0] = 0$ and $C_{m,q}[m-2] = m-1$, so $C_{m,q+1}[0] = H_q[0] = m-1$ and $C_{m,q+1}[m-2] = H_q[m-2] = 0$, as required for $q+1$ (odd);
- if q is odd, then $C_{m,q}[0] = m-1$ and $C_{m,q}[m-2] = 0$, so $C_{m,q+1}[0] = H_q[0] = 0$ and $C_{m,q+1}[m-2] = H_q[m-2] = m-1$, as required for $q+1$ (even).

For $1 \leq i \leq q-1$ we have $C_{m,q+1}[i] = H_q[i] = C_{m,q}[i] = V_i$, and for $q+1 \leq i \leq m-3$ we have $C_{m,q+1}[i] = H_q[i] = C_{m,q}[i] = i$ (the index i takes part neither in the exchange of $\Lambda_{\mathcal{F}_{m-1}}$ nor in the top-level swap). Hence the invariant holds for $q+1$.

Last block ($q = m-2 \to q = m-1$). Here $q = m-2$ is even, $C_{m,m-2}[0] = 0$, $C_{m,m-2}[m-2] = m-1$, and $V_{m-2} = m-3$. Applying the transposition $\widehat{\sigma}_{m,m-1} = (\widehat{m-2\ m-1})$ to H_{m-2} gives

$$C_{m,m-1}[m-2] = H_{m-2}[m-1] = V_{m-2} = m-3,$$
$$V_{m-1} = C_{m,m-1}[m-1] = H_{m-2}[m-2] = C_{m,m-2}[0] = 0.$$

The remaining positions satisfy $C_{m,m-1}[i] = H_{m-2}[i]$: for $i = 0$ we get $H_{m-2}[0] = C_{m,m-2}[m-2] = m-1$; for $1 \le i \le m-3$ we get $H_{m-2}[i] = C_{m,m-2}[i] = V_i$, where $i = 1$ gives $V_1 = m-2$ and $2 \le i \le m-3$ gives $V_i = i-1$. Hence

$$C_{m,m-1} = [\, m-1,\ m-2,\ 1,\ 2,\ \ldots,\ m-4,\ m-3,\ 0\,].$$

Thus $V = (m-1, m-2, 1, 2, \ldots, m-3, 0)$ contains every element of $\{0, 1, \ldots, m-1\}$ exactly once, so $q \mapsto V_q$ is a bijection.

2. Odd order ($m \ge 5$). Here $\widehat{\sigma}_{m,q+1} = (0\ \widehat{m-1})$ is constant for $0 \le q \le m-2$. Expanding $C_{m,q+1} = \widehat{\sigma}_{m,q+1}(\Lambda_{\mathcal{F}_{m-1}}(C_{m,q}))$ entrywise under the source-index convention gives

$$C_{m,q+1}[0] = H_q[m-1] = C_{m,q}[m-1],$$
$$C_{m,q+1}[m-1] = H_q[0] = C_{m,q}\big[\pi_{\mathcal{F}_{m-1}}(0)\big] = C_{m,q}[c_1],$$
$$C_{m,q+1}[i] = H_q[i] = C_{m,q}\big[\pi_{\mathcal{F}_{m-1}}(i)\big] \qquad (1 \le i \le m-2).$$

Since $(c_0, c_1, \ldots, c_{m-2})$ with $c_0 = 0$ is the single-cycle orbit of $\pi_{\mathcal{F}_{m-1}}$ on $\{0, \ldots, m-2\}$, that is, $\pi_{\mathcal{F}_{m-1}}(c_k) = c_{(k+1) \bmod (m-1)}$, while ρ satisfies $\rho(0) = m-1$, $\rho(m-1) = c_1$ and $\rho(c_k) = c_{(k+1) \bmod (m-1)}$ for $1 \le k \le m-2$, a term-by-term comparison yields

$$C_{m,q+1}[i] = C_{m,q}\big[\rho(i)\big] \quad \Longrightarrow \quad C_{m,q+1} = \widehat{\rho}\big(C_{m,q}\big).$$

Starting from $C_{m,0} = I_m$ we obtain $C_{m,q} = \widehat{\rho}^{\,q}(I_m)$, that is, $C_{m,q}[i] = \rho^q(i)$, and hence

$$V_q = C_{m,q}[m-1] = \rho^q(m-1).$$

Since ρ is a single m-cycle on $\{0, 1, \ldots, m-1\}$, the sequence $(V_0, V_1, \ldots, V_{m-1})$ runs through the whole set without repetition, so $q \mapsto V_q$ is a bijection.

The explicit closed form now follows from Theorem 5 at the even order $m-1$,

$$\mathcal{F}_{m-1} = [\, m-4,\ m-3,\ 1,\ 2,\ \ldots,\ m-5,\ m-2,\ 0\,],$$

gives the piecewise form of $\pi_{\mathcal{F}_{m-1}}$: $\pi_{\mathcal{F}_{m-1}}(0) = m-4$, $\pi_{\mathcal{F}_{m-1}}(j) = j-1$ for $2 \le j \le m-4$, $\pi_{\mathcal{F}_{m-1}}(1) = m-3$, $\pi_{\mathcal{F}_{m-1}}(m-3) = m-2$ and $\pi_{\mathcal{F}_{m-1}}(m-2) = 0$. Hence $c_1 = m-4$, and as long as $c_k \in [2, m-4]$ we have $c_{k+1} = c_k - 1$, so $c_{m-4} = 1$; afterwards $c_{m-3} = \pi_{\mathcal{F}_{m-1}}(1) = m-3$, $c_{m-2} = m-2$ and $c_{m-1} = 0$. Substituting into $V = (m-1, c_1, \ldots, c_{m-2}, 0)$ gives the stated closed form. □

5.3 Closed-Form Theorem of Final Arrangements

Theorem 5 (Closed-form theorem of the final arrangement) *After a complete run of the order-m Heap algorithm the final arrangement array $\mathcal{F}_m$ satisfies:*

- ***Base cases:*** $\mathcal{F}_2 = [1, 0]$ *and* $\mathcal{F}_3 = [2, 1, 0]$.
- ***For even*** $m \ge 4$***:*** $\mathcal{F}_m = [m-3,\ m-2,\ 1,\ 2,\ \ldots,\ m-4,\ m-1,\ 0]$.
- ***For odd*** $m \ge 5$***:*** $\mathcal{F}_m = [m-1,\ 1,\ 2,\ \ldots,\ m-2,\ 0]$.

Proof The convergence relation

$$\mathcal{F}_m = \Lambda_{\mathcal{F}_{m-1}}(C_{m,m-1}), \qquad \mathcal{F}_m[i] = C_{m,m-1}\big[\pi_{\mathcal{F}_{m-1}}(i)\big],$$

allows us to project the structure of the block base arrays established in Lemma 4 through the source indices.

Even order. Here $\pi_{\mathcal{F}_{m-1}} = (0\ \ m-2)$ and $C_{m,m-1} = [m-1, m-2, 1, \ldots, m-4, m-3, 0]$. Expanding entrywise,

$$\mathcal{F}_m[0] = C_{m,m-1}[m-2] = m-3, \qquad \mathcal{F}_m[1] = C_{m,m-1}[1] = m-2,$$

while for $2 \le i \le m-3$ we have $\pi_{\mathcal{F}_{m-1}}(i) = i$ and hence $\mathcal{F}_m[i] = C_{m,m-1}[i] = i-1$; the boundary terms give $\mathcal{F}_m[m-2] = C_{m,m-1}[0] = m-1$ and $\mathcal{F}_m[m-1] = C_{m,m-1}[m-1] = 0$. Therefore

$$\mathcal{F}_m = [\, m-3,\ m-2,\ 1,\ 2,\ \ldots,\ m-4,\ m-1,\ 0\,].$$

Odd order. Here $C_{m,m-1} = \widehat{\rho}^{\,m-1}(I_m) = \widehat{\rho}^{\,-1}(I_m)$, that is, $C_{m,m-1}[i] = \rho^{-1}(i)$. Using the cycle table of ρ, namely $\rho(0) = m-1$, $\rho(m-1) = c_1$ and $\rho(c_k) = c_{(k+1) \bmod (m-1)}$,

$$\mathcal{F}_m[0] = C_{m,m-1}\big[\pi_{\mathcal{F}_{m-1}}(0)\big] = \rho^{-1}(c_1) = m-1,$$

while for $1 \le i \le m-2$, writing $i = c_k$, we have $\pi_{\mathcal{F}_{m-1}}(i) = c_{(k+1) \bmod (m-1)}$ and hence $\mathcal{F}_m[i] = \rho^{-1}(c_{(k+1) \bmod (m-1)}) = c_k = i$; the trailing entry satisfies $\mathcal{F}_m[m-1] = C_{m,m-1}[m-1] = \rho^{-1}(m-1) = 0$. Hence $\mathcal{F}_m = [\, m-1, 1, 2, \ldots, m-2, 0\,]$. □

5.4 Dichotomy of Permutation Cycle Structures

Lemma 6 (Dichotomy of the cycle structure of the final arrangement) *Let $m \ge 2$. The induced permutation $\pi_{\mathcal{F}_m} \in S_m$ of the final arrangement (that is, $\pi_{\mathcal{F}_m}(i) = \mathcal{F}_m[i]$) has the following strict dichotomy.*

- ***Even order*** *($m = 2k \ge 4$).* *$\pi_{\mathcal{F}_m}$ is a single m-cycle on $\{0, 1, \ldots, m-1\}$, with piecewise form*

  $$\pi_{\mathcal{F}_m}(0) = m-3, \qquad \pi_{\mathcal{F}_m}(1) = m-2, \qquad \pi_{\mathcal{F}_m}(2) = 1,$$
  $$\pi_{\mathcal{F}_m}(i) = i-1 \ \ (3 \le i \le m-3), \qquad \pi_{\mathcal{F}_m}(m-2) = m-1, \qquad \pi_{\mathcal{F}_m}(m-1) = 0,$$

  and correspondingly the orbit chain can be written as

  $$0 \ \mapsto\ m-3 \ \mapsto\ m-4 \ \mapsto\ \ldots \ \mapsto\ 2 \ \mapsto\ 1 \ \mapsto\ m-2 \ \mapsto\ m-1 \ \mapsto\ 0,$$

  where $m-3 \mapsto m-4 \mapsto \cdots \mapsto 2$ denotes the consecutive segment decreasing from $m-3$ to 2. This segment is empty for $m = 4$, where the orbit reads $0 \mapsto 1 \mapsto 2 \mapsto 3 \mapsto 0$; the piecewise form above is the authoritative statement.
- ***Odd order*** *($m = 2k+1 \ge 3$).* *$\pi_{\mathcal{F}_m}$ is exactly the transposition $(0\ \ m-1)$, so $\pi^2_{\mathcal{F}_m} = \mathrm{id}$.*

Proof We verify algebraically with the closed form of Theorem 5. For even m the closed form $\mathcal{F}_m = [m-3, m-2, 1, 2, \ldots, m-4, m-1, 0]$ gives the piecewise form stated above by reading the entries off one by one. From $\pi_{\mathcal{F}_m}(i) = i-1$ for $3 \le i \le m-3$ we obtain the decreasing segment $m-3 \to m-4 \to \cdots \to 3 \to 2$ (empty when $m = 4$), and then $\pi_{\mathcal{F}_m}(2) = 1$, $\pi_{\mathcal{F}_m}(1) = m-2$, $\pi_{\mathcal{F}_m}(m-2) = m-1$ and $\pi_{\mathcal{F}_m}(m-1) = 0$ return to the starting point. The orbit passes through all m elements and contains no proper sub-cycle, so it is a single m-cycle.

For odd m the closed form $\mathcal{F}_m = [m-1, 1, \ldots, m-2, 0]$ gives $\pi_{\mathcal{F}_m}(0) = m-1$ and $\pi_{\mathcal{F}_m}(m-1) = 0$, while $\pi_{\mathcal{F}_m}(i) = i$ for $1 \le i \le m-2$; hence $\pi_{\mathcal{F}_m} = (0\ \ m-1)$, whose square is the identity. □

5.5 Completeness of the Unrank Algorithm

Theorem 7 (Completeness of the Unrank algorithm) *For any valid code $d = (d_0, \ldots, d_{n-2})$ the output of the Unrank algorithm is exactly the permutation of the Heap output sequence whose index is*

$$r(d) := \sum_{i=0}^{n-2} d_i \cdot (i+1)!,$$

that is, $\mathrm{Unrank}_n(d) = A^{(n)}_{r(d)}$.

Proof Induct on the dimension m and prove the stronger level-by-level statement

$$\mathrm{Unrank}_m(d_0, \ldots, d_{m-2}) = A^{(m)}_{r_m}, \qquad r_m := \sum_{i=0}^{m-2} d_i \cdot (i+1)!.$$

Base cases ($m \le 2$). For $m = 1$ the algorithm returns $[0] = A^{(1)}_0$. For $m = 2$ the code space is $\{d_0\}$ and $r_2 = d_0 \cdot 1! = d_0 \in \{0, 1\}$; the algorithm returns $[0, 1]$ or $[1, 0]$ according to d_0, while $A^{(2)}_0 = [0, 1]$ and $A^{(2)}_1 = [1, 0]$. The claim holds.

Inductive step. Assume the claim for all orders $< m$. Write $q := d_{m-2} \in \{0, \ldots, m-1\}$ for the block index, $r_{m-1} := \sum_{i=0}^{m-3} d_i(i+1)!$, and $A_{m-1} := \mathrm{Unrank}_{m-1}(d_0, \ldots, d_{m-3})$; by the induction hypothesis $A_{m-1} = A^{(m-1)}_{r_{m-1}}$.

By Definition 8,

$$\mathrm{Unrank}_m(d) = \Lambda_{A_{m-1}}\big(B_{m,q}(I_m)\big).$$

By Lemma 2, $B_{m,q}(I_m) = C_{m,q} = A^{(m)}_{q\,(m-1)!}$ is the block base array of the q-th block, and the k-th output inside that block satisfies

$$A^{(m)}_{q\,(m-1)!+k} = E_m\big(\widehat{\pi_{A^{(m-1)}_k}}\big)\big(C_{m,q}\big) \qquad (0 \le k \le (m-1)! - 1).$$

Taking $k = r_{m-1} \le (m-1)! - 1$ and noting that $\Lambda_{A_{m-1}} = E_m(\widehat{\pi_{A_{m-1}}})$, we get

$$\mathrm{Unrank}_m(d) = E_m\big(\widehat{\pi_{A^{(m-1)}_{r_{m-1}}}}\big)\big(C_{m,q}\big) = A^{(m)}_{q\,(m-1)!+r_{m-1}}.$$

The mixed-radix expansion

$$q \cdot (m-1)! + r_{m-1} = d_{m-2}(m-1)! + \sum_{i=0}^{m-3} d_i(i+1)! = \sum_{i=0}^{m-2} d_i(i+1)! = r_m$$

gives $\mathrm{Unrank}_m(d) = A^{(m)}_{r_m}$. □

5.6 Reciprocal Bijection between Rank and Unrank

Theorem 8 (Reciprocal bijection between Rank and Unrank) *For any valid code $d = (d_0, \ldots, d_{n-2})$ and any Heap permutation perm of order n,*

$$\mathrm{Rank}\big(n, \mathrm{Unrank}(n, d)\big) = d, \qquad \mathrm{Unrank}\big(n, \mathrm{Rank}(n, \mathit{perm})\big) = \mathit{perm}.$$

Proof

1. Since the values in the sequence are distinct, each trailing value corresponds to a unique sub-block index, so the Rank loop determines the code digit uniquely.
2. Since S_q^{-1} is the strict inverse of the position–value mapping (Proposition 3), the array remaining after the trailing element has been stripped off is exactly a lower-order Heap sub-permutation.

After reducing layer by layer down to the base level, the reverse process reproduces the forward expansion steps of Unrank exactly. Every step is invertible, so the composition of the two restores the original input. □

5.7 Bijectivity and Generation Index Consistency

Theorem 9 (Bijectivity and generation index consistency) Unrank_n *is a bijection between the code space* $\prod_{i=0}^{n-2}\{0,1,\ldots,i+1\}$ *and the full permutation space* Arr_n*; moreover the global generation index of the code* d *in the Heap output sequence is given by the closed form*

$$r(d) = \sum_{i=0}^{n-2} d_i \cdot (i+1)!,$$

with maximum index $r_{\max} = n! - 1$*. The closed form also shows that the Heap generation order is the* ***colex order*** *of the code vectors* d *(see Remark 2).*

Proof **(i) Bijectivity.** Induct on the size m; the case $m = 2$ is clear. Assume Unrank_{m-1} is a bijection. At level m the high-order digit $d_{m-2} \in \{0,1,\ldots,m-1\}$ corresponds to m mutually disjoint blocks.

For a fixed block $q = d_{m-2}$ the lower-order operator group acts without changing the trailing position and the array entries are distinct, so the map $P \mapsto \widehat{P}(A)$ is injective on S_{m-1}; the block therefore contains exactly $(m-1)!$ distinct permutations.

By Lemma 4 every permutation in the q-th block has trailing element constantly V_q, and $q \mapsto V_q$ is a bijection of $\{0,\ldots,m-1\}$, so permutations generated in different blocks have different trailing elements and the blocks are disjoint.

The total image has size $m \times (m-1)! = m! = |\,\mathrm{Arr}_m\,|$, so Unrank_m is a bijection.

(ii) Generation index consistency. This is immediate from Theorem 7: for every valid code d we have $\mathrm{Unrank}_n(d) = A^{(n)}_{r(d)}$ with $r(d) = \sum_{i=0}^{n-2} d_i \cdot (i+1)!$, so $r(d)$ is exactly the generation index of d in the Heap output sequence.

Since r is the mixed-radix expansion of the digits with weights $1!, 2!, \ldots, (n-1)!$ and the d_i range independently over $\{0,\ldots,i+1\}$,

$$r : \prod_{i=0}^{n-2}\{0,1,\ldots,i+1\} \longrightarrow \{0,1,\ldots,n!-1\}$$

is a bijection; the maximum is attained when every digit takes its upper bound $d_i = i+1$,

$$r_{\max} = \sum_{i=0}^{n-2}(i+1)(i+1)! = n! - 1.$$

(The last equality is the identity $\sum_{k=1}^{n-1} k \cdot k! = n! - 1$.) □

Corollary 10 (The Heap order is the colex order of the code vectors) *Ordering the code space* $\prod_{i=0}^{n-2}\{0,1,\ldots,i+1\}$ *by increasing generation index yields exactly the* ***colex order*** *of the code vectors* d *(with the last coordinate* d_{n-2} *as the most significant); equivalently, the Heap generation order is the colex order of the code* d*, not its lexicographic order.*

Proof By Theorem 9 the generation index $r(d) = \sum_{i=0}^{n-2} d_i \cdot (i+1)!$ is the mixed-radix expansion whose weights $1!, 2!, \ldots, (n-1)!$ increase from left to right. As r increases, the digit of lowest weight, d_0, changes first, then d_1, and so on, while the digit of highest weight, d_{n-2} — the last coordinate of the vector — changes last, which is the definition of colex order. □

6 Numerical Verification

The tables below collect the small-order cases in closed form. The algorithms of Section 4 are implemented in C++ together with a test suite that checks every code of order $n \leq 10$ against the iterative algorithm of Section 2.1; that implementation is available at [9].

6.1 Complete Case Study for $n = 3$

The case $n = 3$ is the odd-order base case; its mixed-radix code $d = (d_0, d_1)$ has base vector $(1!, 2!)$. Table 1 lists the code, the generation index, the block base array and the Unrank result for all 6 states.

Code (d_0, d_1)	Index $r(d) = d_0 \cdot 1! + d_1 \cdot 2!$	Block base C_{3,d_1}	Result $\mathrm{Unrank}_3(d)$
$(0, 0)$	0	$[0, 1, 2]$	$[0, 1, 2]$
$(1, 0)$	1	$[0, 1, 2]$	$[1, 0, 2]$
$(0, 1)$	2	$[2, 0, 1]$	$[2, 0, 1]$
$(1, 1)$	3	$[2, 0, 1]$	$[0, 2, 1]$
$(0, 2)$	4	$[1, 2, 0]$	$[1, 2, 0]$
$(1, 2)$	5	$[1, 2, 0]$	$[2, 1, 0]$

Table 1 Complete case study for $n = 3$: the third column gives the exact block base array C_{3,d_1} and the last column the Unrank result.

6.2 $n = 4$: Even-Order Intermediate States and Block Base Arrays

The case $n = 4$ is of even order. To display the evolution of the even-order intermediate structure, Table 2 records the high-order block base arrays $C_{4,q}$ together with their trailing elements V_q, and distinguishes at the end the terminal block base array $C_{4,3}$ from the final arrangement $\mathcal{F}_4$.

Block index $q = d_2$	Block base $C_{4,q}$	Trailing V_q	Covered code range and state
$q = 0$	$[0, 1, 2, 3]$	3	$(0, 0, 0) \sim (1, 2, 0)$, 6 permutations
$q = 1$	$[3, 1, 0, 2]$	2	$(0, 0, 1) \sim (1, 2, 1)$, 6 permutations
$q = 2$	$[0, 2, 3, 1]$	1	$(0, 0, 2) \sim (1, 2, 2)$, 6 permutations
$q = 3$	$[3, 2, 1, 0]$	0	$(0, 0, 3) \sim (1, 2, 3)$; terminal block base $C_{4,3}$, final state $\mathcal{F}_4 = [1, 2, 3, 0]$

Table 2 $n = 4$: evolution of the block base arrays $C_{4,q}$ and the trailing sequence $V = (3, 2, 1, 0)$ (the even-order case of Lemma 4).

6.3 $n = 5$: Odd-Order Single-Cycle Orbit Verification

To further verify the derivation of the odd-order single-cycle orbit operator $C_{m,q} = \widehat{\rho}^{\,q}(I_m)$, Table 3 gives the complete 5-entry block base arrays $C_{5,q}$ and the trailing-element trace for $m = 5$, showing the orbit length and its bijectivity.

$q = d_3$	Block base $C_{5,q}$	$V_q = C_{5,q}[4]$	Orbit check
$q = 0$	$[0, 1, 2, 3, 4]$	4	$V_0 = 4$ (reference)
$q = 1$	$[4, 2, 3, 0, 1]$	1	$V_1 = \rho(4) = 1$
$q = 2$	$[1, 3, 0, 4, 2]$	2	$V_2 = \rho^2(4) = 2$
$q = 3$	$[2, 0, 4, 1, 3]$	3	$V_3 = \rho^3(4) = 3$
$q = 4$	$[3, 4, 1, 2, 0]$	0	$V_4 = \rho^4(4) = 0$ (terminal)

Table 3 $n = 5$: the odd-order trailing sequence $V = (4, 1, 2, 3, 0)$ and verification of the single-cycle orbit operator $C_{5,q} = \widehat{\rho}^{\,q}(I_5)$.

7 Complexity Analysis

7.1 Time Complexity

Both algorithms traverse the $n-1$ levels $k = 2, \ldots, n$ and advance at most $k-1$ times per level. One advance first writes down Step$(k-1)$ from the closed form ($O(k)$) and then rearranges the mapping of length $k-1$ according to that table and swaps one entry ($O(k)$); each level also rewrites a permutation of length $O(k)$. The cost of level k is therefore $O(k^2)$ and the total work is

$$\sum_{k=2}^{n} O(k^2) = O(n^3).$$

The bound is **tight**: a single advance at level k already costs $\Theta(k)$, so Unrank runs in $\Theta(n^3)$ time. For Rank the inverse mapping $p \leftarrow [S^{-1}(p[i])]$ at the end of each level is itself $\Theta(k^2)$, so Rank is $\Theta(n^3)$ as well. The constant factors of both are small, and neither uses hash tables, binary search or tree structures.

Table 4 compares the approach with existing schemes.

Table 4 Comparison of permutation Rank/Unrank schemes. All listed schemes use $O(n)$ space and need no precomputation. The present method is slower in time; in exchange it supports the Heap generation order *directly*.

Scheme	Generation order	Rank/Unrank time
Lexicographic (list shifting)	lexicographic	$\Theta(n^2)$
Lexicographic (Fenwick tree)	lexicographic	$\Theta(n \log n)$
Myrvold–Ruskey [6]	Fisher–Yates order	$\Theta(n)$
Present method	**Heap order**	$\Theta(n^3)$

The $\Theta(n)$ of Myrvold–Ruskey and the $\Theta(n^3)$ obtained here are not measured on the same problem: the former addresses an arbitrarily associated mixed-radix order, which has no direct counterpart in the block-transition structure of the Heap order (Section 1). If one merely needs *some* generation order that can be ranked in linear time, the former should be used directly; the value of the present method is that it covers the Heap order itself and needs no factorial-scale lookup table. Reducing $\Theta(n^3)$ to $O(n^2)$ or $O(n \log n)$ is left for future work, for instance by maintaining the position mapping incrementally instead of rebuilding it at every level.

7.2 Space Complexity

The whole process stores only three linear arrays: the position mapping, the working permutation and the mixed-radix code d, all of length at most n. There is no factorial-scale precomputed table and no recursion-depth overhead. In the iterative implementation the memory grows only linearly with n, so the space complexity is $O(n)$.

7.3 Pre-computation Overhead

The algorithms require no offline precomputation. Block trailing values and rearrangement transformations are produced online in real time, so no sequence or block-mapping table has to be stored; the factorial-scale blow-up in space is avoided.

8 Conclusion

This paper has described the block structure of Heap's permutation generation order through the rearrangement operators that realise it, and has used that description to obtain a Rank/Unrank pair for the Heap order. With the source-index convention fixing the composition direction, a block-internal traversal and a block-boundary interchange are the tail-preserving operator and the top-level swap operator, and the block base arrays satisfy a closed-form recurrence; both algorithms work from that recurrence, need no precomputed table, and use $O(n^3)$ time and $O(n)$ space. Their correctness rests on the induction of Section 5, and the cases $n = 3, 4, 5$ of Section 6 check the resulting closed forms against explicit values.

Declarations

- **Funding:** No funding was received for this study.
- **Competing interests:** The author declares no competing interests.
- **Data availability:** All data generated or analysed during this study are included in this published article.
- **Code availability:** The reference implementation of the algorithms and its test suite are available at https://github.com/Yusheng-Hu/Heaps-unrank-rank.

References

[1] H. F. Trotter. Algorithm 115: Perm. *Communications of the ACM*, 5(8):434–435, 1962.

[2] B. R. Heap. Permutations by interchanges. *The Computer Journal*, 6(3):293–294, 1963.

[3] S. M. Johnson. Generation of permutations by adjacent transposition. *Mathematics of Computation*, 17(83):282–285, 1963.

[4] F. Ives. Permutation enumeration: four new permutation algorithms. *Communications of the ACM*, 19(2):68–72, 1976.

[5] Robert Sedgewick. Permutation generation methods. *Computing Surveys*, 9:137–164, 1977.

[6] Wendy Myrvold and Frank Ruskey. Ranking and unranking permutations in linear time. *Information Processing Letters*, 79(6):281–284, 2001.

[7] F. Ruskey. *Combinatorial Generation*. Working draft, University of Victoria, 2003.

[8] Donald E. Knuth. *The Art of Computer Programming, Volume 4, Fascicle 2: Generating All Tuples and Permutations*. Addison-Wesley Professional, Reading, MA, 2005.

[9] YUSHENG HU. Reference implementation of the Heap permutation rank and unrank algorithms. GitHub repository, 2026. https://github.com/Yusheng-Hu/Heaps-unrank-rank.